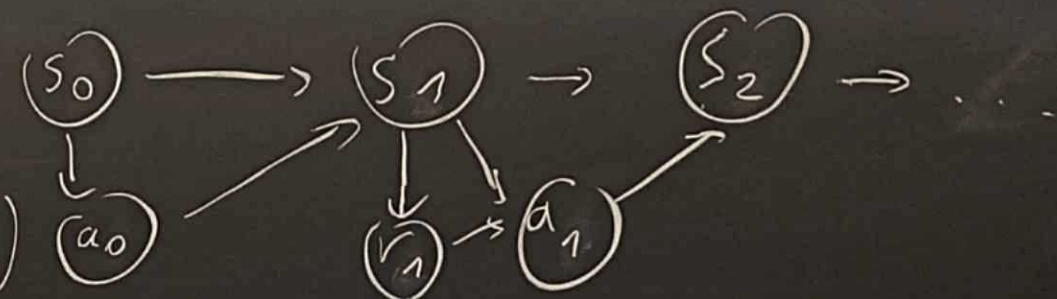


Reinforcement learning (world)

- world is a fully observable Markov decision process:
by Markov property (future only depends on present, not past)

this can be described by only 2 probabilities: world evolution



$p^*(s', r | s, a)$ what happens when action a is done in state s
new state $\uparrow$ $\uparrow$ reward

- return: total reward, possibly discounted

agent's policy: $\pi(a | s)$ how to select action in state s

$$R = \sum_{t=1}^{\infty} \gamma^{t-1} r_t$$

γ discount factor $\gamma \in (0, 1]$

∞ finite games: when game ends at $t=T$
rewards $r_{t>T} = 0 \Rightarrow$ auto truncation

- goal: choose policy π^* that maximises the expected return after infinitely many games

infinite games: truncate when $\gamma^{t-1} \approx 0$

• value function: $V_{\pi}(s) = \mathbb{E}_{\text{all games played with policy } \pi, \text{ starting in state } s \text{ at } t=0} [R]$ return of a single episode

• Bellman equations: $V_{\pi}(s) = \mathbb{E}_{a \sim \pi(a|s)} \left[\mathbb{E}_{s', r \sim p^*(s', r|s, a)} \left[\underbrace{r}_{\text{immediate reward}} + \underbrace{\gamma V_{\pi}(s')}_{\text{discounted return of remainder of episode}} \right] \right]$

if s and a are finite (= discrete), this is a linear system of equations to be solved by value iteration

• catch: in practice, we do not know p^* and want to avoid learning $\hat{p} \approx p^*$

• introduce action-value function: $Q_{\pi}(s, a) = \mathbb{E}_{s', r \sim p^*} [r + \gamma V_{\pi}(s')] \quad \begin{array}{l} \text{play action } a \text{ in state } s \text{ and} \\ \text{use policy } \pi \text{ afterwards} \end{array}$

$$V_{\pi}(s) = \mathbb{E}_{a \sim \pi(a|s)} \left[\underbrace{Q_{\pi}(s, a)} \right]$$

if unknown $\Rightarrow$ 2 approximations

{ Q-learning: $V_{\pi}(s) = \max_a Q_{\pi}(s, a)$

{ SARSA: $V_{\pi}(s) = Q_{\pi}(s, a_{TS})$ expectation $\approx$ a single instance from TS

Bellman equations for Q-function: (with Q-learning rule)

$$Q_{\pi}(s, a) = \mathbb{E}_{s' \sim p \times} \left[r + \gamma \max_{a'} Q(s', a') \right]$$

Theorem: if this is iterated with an infinite TS, it converges to the optimal Q-function $Q^* = Q_{\pi^*}$

• derive policy from Q-function: - deterministic: $\pi(a|s) = \delta(a - \pi(s))$, $\pi(s) = \arg \max_a Q(s, a)$

"greedy": may result in poor long-term returns when Q is not optimal

- ϵ -greedy: perform a random action with probability ϵ

$$\pi(a|s) = (1 - \epsilon) \pi(s) + \epsilon \cdot \text{uniform}(a) \quad (\text{popular during training})$$

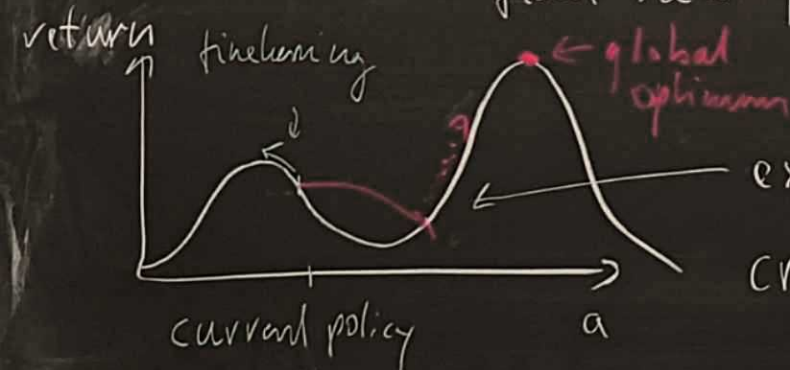
- softmax: $\pi(a|s) = \text{softmax}_{\beta}(Q(s, a))$ β : inverse temperature (hyperparameter)

$$\beta \rightarrow 0: \text{softmax}_0(Q) = \text{uniform}(a)$$

$$= \exp(\beta \cdot Q(s, a)) / \sum_{a'} \exp(\beta \cdot Q(s, a'))$$

$\beta \rightarrow \infty$: greedy

- during training, we must balance exploration vs. exploitation
 - find new promising actions $\uparrow$
 - fine tune your current policy $\downarrow$



exploration: initially worse, but may converge to a better solution
 crucial for training: do enough of both $\Rightarrow$ adjust ϵ or β accordingly

- temporal difference learning: learn Q -function from a TS

$$Q^*(s,a) = \underbrace{Q^{(t-1)}(s,a)}_{\text{current guess}} + \left[\underbrace{Q^*(s,a)}_{\text{ideal}} - Q^{(t-1)}(s,a) \right]$$

$\approx R$ from a specific game
 starting at $\bar{s}$
 (empirical average with 1 term)

$\Rightarrow$ iterative update rule: $Q^{(\tau)}(s,a) = Q^{(\tau-1)}(s,a) + \alpha \cdot [Q^*(s,a) - Q^{(\tau-1)}(s,a)]$
new guess $\uparrow$ learning rate

$$Q^{(\tau)}(s,a) \approx Q^{(\tau-1)}(s,a) + \alpha [r + \gamma V^{(\tau-1)}(s') - Q^{(\tau-1)}(s,a)]$$

where (s, a, r, s') are from TS

and $V^{(\tau-1)}(s') = \max_{a'} Q(s', a')$ or $V^{(\tau-1)}(s') = Q^{(\tau-1)}(s', a')$
Q-learning rule (TS: (s, a, r, s', a')) SARSA

full Q-learning algorithm:

⑥ define initial guess $Q^{(0)}(s, a)$, such that $Q^{(0)}(s = \text{terminal state}, a) = 0$

⑦ outer loop: $\tau = 1, \dots, \tau_{\max}$ (training epochs)

① inner loop: $t = 1, \dots, T_\tau$ (steps of a single episode)

(I) choose action by ϵ -greedy or softmax policy, using current guess $Q^{(\tau-1)}(s, a)$ for each s

(II) add all observed transitions (s, a, r, s', a') to TS

② sample a batch from TS and use temporal difference formula to update $Q^{(\tau)}(s, a)$

⑧ when converged to $\hat{Q} = Q^{(\tau_{\max})}$, define final policy (greedy or softmax with large β)

how to sample a batch: - do not pick multiple transitions from same game $\Rightarrow$ highly correlated $\Rightarrow$ overfitting!
- curriculum learning: gradually make the batches harder
- weighted sampling: prefer transitions where temporal difference $R - Q^{(\tau-1)}$ is big

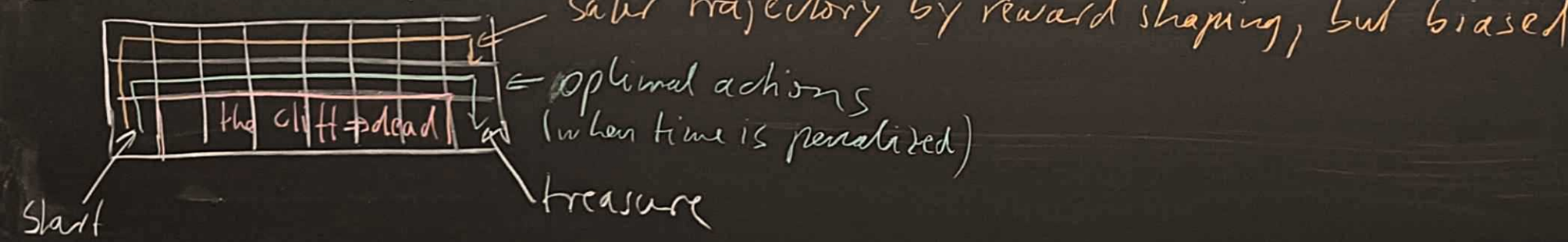
2 possibilities to set-up regression model:

- element-wise regression: $Q(s, a) \approx f_{\theta}(s, a)$
- multi-head regression: $\underbrace{Q(s)}_{\text{column for all } a} = f_{\theta}(s)$ entire column of Q at once

• for example, linear regression: - define hand-crafted features $\tilde{x} = \phi(s)$ (crucial!)

- one weight vector per action: $Q(s, a) \approx \phi(s) \cdot \beta_a \Rightarrow$ learn all β_a using $(R_t - \phi(x) \beta_a)^2$

Reward shaping



2 solutions: (1) reward shaping schedule: reduce auxiliary rewards as agent gets better

(2) auxiliary rewards are a "potential" $\tilde{r}_{t+1} \leftarrow r_{t+1} + \gamma \psi(s_{t+1}) - \gamma \psi(s_t)$

$\Rightarrow$ theorem: this does not bias Q

$\Rightarrow$ reward shaping is useful, but must be done with care